

NYU Tandon School of Engineering
Spring 2026, ECE 9953 (Dr. Karri) - Independent Project
Using LLMs for UVM Testbench Generation
Project Report

Student Name: Archie Gupta
Net ID: ag10276

Introduction

Functional verification is the dominant cost in modern ASIC and SoC development, accounting for an estimated 60–70% of total project effort. Within that budget, the Universal Verification Methodology (UVM) , a SystemVerilog class library standardized by IEEE 1800.2, is the dominant industrial framework. A complete UVM testbench (TB) for even a single design comprises roughly eleven coordinated components: sequence item, sequence, sequencer, driver, monitor, agent, scoreboard, functional coverage subscriber, environment, test, and a top-level harness. Most of this is structural boilerplate: the same factory-registration macros, the same phase methods, the same TLM hookups, instantiated across every design under test (DUT).

The pattern-driven nature of UVM testbench construction makes it a natural target for large language models. Recent work has shown LLMs can produce non-trivial RTL and assertion-style properties. The narrower question of whether LLMs can produce production-quality UVM testbenches remains open: producing eleven files with the right names and the right macros is not the same as producing a TB that elaborates, simulates, and meaningfully exercises the DUT.

This report contributes a controlled benchmark answering that question. I compare three LLMs models across 14 RTL designs of varying complexity, with a hand-written golden testbench for each design serving as the reference. All testbenches are evaluated under an identical Synopsys VCS pipeline with the same SystemVerilog Assertion (SVA) module bound to each DUT. I checked the generated output against four evaluation criteria: (i) structural conformance, (ii) compile and runtime success, (iii) functional coverage closure, and (iv) assertion coverage. I further classify all observed failure modes into a three-bucket taxonomy that scopes the cost of fixing each.

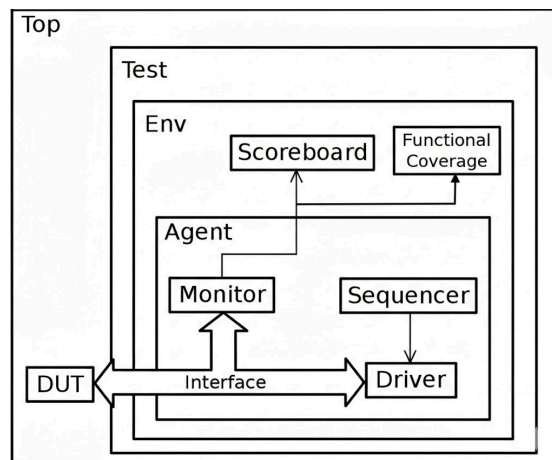


Image 1: UVM Testbench Structure

Methodology

Benchmark designs

I chose fourteen RTL designs spanning three difficulty tiers. Combinational designs: 4:1 multiplexer, 3-to-8 one-hot decoder, 2-bit comparator, 4-bit ripple-carry adder, 8-bit ALU (8 opcodes), and 8-bit right barrel shifter. Sequential designs: 4-bit up/down counter, 8-bit universal shift register (USR), four-state traffic-light FSM, round-robin arbiter, and 8-entry synchronous FIFO. Bus-protocol slaves: APB3 memory slave, AHB-Lite memory slave and AXI4 memory slave. For each design I wrote a complete UVM testbench (the reference), bound a SystemVerilog Assertion module to the DUT, and authored both directed and randomised stimulus.

Models and generation pipeline

Three LLMs were evaluated: o4-mini, Gemini Pro 3.1 Preview, and Claude Opus 4.6. A Python driver loads the RTL/Specification, builds a structured prompt that lists the required UVM sections and embeds the DUT module, calls the model through a unified API gateway, and parses the response into the eleven UVM files plus a top-level harness, a top-level Makefile, and a filelist. Each (model, design) pair was attempted once; failing pairs were retried up to three additional times with the same prompt to bound retry-recovery rates.

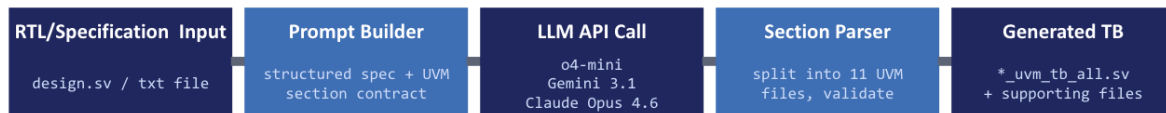


Image 2: Framework for LLM-Assisted TB Generation

Evaluation

All testbenches, both LLM-generated and golden, were compiled and simulated under Synopsys VCS with identical compile flags and the same bound SVA module. I measured: (i) structural conformance, whether all eleven required UVM sections are produced; (ii) compile and runtime success, recording every manual edit required to reach a clean elaboration and a non-hanging simulation; (iii) functional coverage, reading the coverage report to compute variable-bin closure, cross-bin closure, and total covered bins; and (iv) assertion coverage, recording per-property attempts, matches, and pass/fail status. Compile failures were tagged with one of 30 issue classes grouped into three buckets (prompt-fixable, architectural, model-inherent). On the coverage side, I additionally classified each design's outcome by which testbench is best on three sub-axes: best by raw score, best from a testing point of view (depth of model and discipline), and best on assertion coverage (stimulus density and pass rate).

Results

Using LLMs to close coverage gaps using directed testing

First-pass simulation of any of the 14 testbenches in this study, including the hand-written golden testbenches, rarely produces 100% coverage closure. Random stimulus tends to hit the easy interior of the coverage space and miss boundary bins, asymmetric crosses, and temporally-conditioned cover properties (back-to-back transitions, load-then-hold sequences, write-when-full guards). The conventional response is for the verification engineer to read the URG report by hand, identify the uncovered bins, and write a directed sequence that targets each gap. I instead used the LLM as the closure engine.

Metric	Run 1 (Random)	Run 2 (Directed)	Combined
Stimuli count	832	104	936
User assertions	32/32 ✓	32/32 ✓	32/32 ✓
Coverpoints	34/34 (100%)	34/34 (100%)	34/34 (100%)
Cross coverage	410/512 (80.08%)	104/512 (20.31%)	512/512 (100%)
Uncovered bins	102	408	0

CLOSURE ACHIEVED ✓ The two test runs are perfectly complementary. Run 1 covered 410 bins, Run 2 covered the remaining 104 bins. Together: $410 + 104 = 514$ unique bins covered out of 512 expected (overlap of 2 bins). Combined coverage: 100%.

Image 3: *Adder* - Using LLMs to generate directed tests for coverage closure

Three patterns emerge across the closure runs. First, the LLM is most effective on designs whose uncovered bins are reachable through a small, well-described modification to the constraint solver, widening a value range, adding a randc cycling order, or specifying a directed (a, b, cin) tuple list. Second, the LLM is also effective at proposing modelling changes the human author had missed.

Design	Pass 1 score	Final score	LLM passes added	Closure mechanism
Mux	65.28%	100.00%	1	Constraint widening on input data values
Decoder	100.00%	100.00%	0	Closed organically (small input space)
Comparator	100.00%	100.00%	0	Closed organically
Adder	80.08%	100.00%	1	Directed sequence targeting 102 uncovered (a, b, cin) bins
ALU	100.00%	100.00%	0	Closed organically (opcode-only model)
Barrel shifter	100.00%	100.00%	0	Closed organically (shift-amount-only model)
Counter	—	100%	1	Coverage subscriber missing in original TB; LLM added covergroup spec
USR	83.33%	100.00%	1	Directed load-then-hold sequence to fire COVER_LOAD_THEN_HOLD
FSM	100.00%	100.00%	0	Closed organically
Sync FIFO	100.00%	100.00%	0	Closed organically; LLM suggested data/transition extensions

Design	Pass 1 score	Final score	LLM passes added	Closure mechanism
APB	100.00%	100.00%	0	Closed organically across 3 covergroups
Arbiter	<100%	100.00%	1	LLM added illegal bins for impossible req/grant pairs (60 → 32 legal cross)
AHB-Lite	70.56%	70.56%	1	LLM added INCR4 burst sequence; IDLE/BUSY HTRANS still uncovered
AXI4	65.52%	65.52%	1	LLM authored back-pressure pass; 8 stability bins remain unreachable (DUT architecture)

Table 1: LLM-driven coverage closure outcomes per design. First-pass coverage score, final score after LLM-driven feedback iterations, number of additional LLM passes added, and the closure mechanism the model proposed.

Structural conformance

Table 2 reports first-attempt structural conformance for all three models. Claude Opus 4.6 and o4-mini tied at 93% (13 of 14 designs each); Gemini Pro 3.1 Preview reached only 29%. Failure modes correlate with output-token budget rather than model size:

	adder	ahb	alu	apb	arbiter	axi	barrel	comp	counter	decoder	fifo	fsm	mux	syncfifo
o4-mini	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Gemini 3.1	✓	✗	✗	✗	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
Claude Opus 4.6	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓

Image 4: Breakdown of Design TB vs LLM Model for first pass success

Gemini 3.1 fails by truncation on the long protocol-design TBs, which was resolved by increasing the tokens permitted for the attempt from 16k to 63k. Retries recovered structural conformance for every failure on at least one of three attempts, indicating the underlying capability is present but determinism is the gap.

Model	Pass rate	Designs passed	Primary failure mode
o4-mini	93%	13 / 14	AHB only (incidental compile error)
Gemini Pro 3.1	29%	4 / 14	Heavy output truncation
Claude Opus 4.6	93%	13 / 14	AXI only (output budget)

Table 2: First-attempt structural conformance. Percentage of 14 designs for which the model produced all eleven required UVM sections in a single attempt. Passes/failures by design and the dominant failure mode are listed.

Compile and runtime success

Zero of three models produced a testbench that compiled and ran cleanly under VCS without manual intervention. Every output required at least one human edit; some required dozens.

Issue	Error Category	Fix
missing `uvm_object_utils(counter_seq_item)` and logic [3:0] dout;	A	Add factory macro + output field
body declared as function	A	Change to virtual task body()
start(item) used incorrectly	C	Replace with start_item(item) / finish_item(item)
invalid uvm_test_top.clock cross-module reference	A	Removed
coverage class samples vif.cb_mon.* (compile fail)	A	Replace with transaction-based coverage
argument name mismatch in coverage write()	C	Rename item to t
sequence declaration / assignment combined → syntax error	C	Split into two statements
run declared as function	A	Change to virtual task run_phase(uvm_phase phase)
starting sequence inside function	B	Move into run_phase task
driver's seq_item_port not connected to sequencer	B	Add driver.seq_item_port.connect(sequencer.seq_item_export)

Image 4: **Counter TB** generated using Claude - Compile Errors and their Categories

I classified the resulting issue catalogue (Table 3) into three buckets: A , Prompt-Fixable (15 issue classes, hundreds of instances), B , Architectural (6 issue classes), and C , Model-Inherent (9 issue classes). Bucket A dominates by frequency: missing `uvm\_macros.svh`, missing factory-registration macros, concrete (non-virtual) interface declarations, run_phase declared as a function rather than a task, and coverpoints that sample the virtual interface instead of the transaction. Bucket B captures architectural misunderstandings: components created in the wrong UVM phase, missing connect_phase wiring, and forgotten objections that cause PH_TIMEOUT or simulation hangs. Bucket C is the residual that prompting cannot close: hallucinated UVM APIs (sequencer.start(), driver.start()), missing transaction output fields, and wrong scoreboard reference models that compile cleanly but flag every transaction with [SCB_FAIL]. Gemini 3.1 was the model most prone to wrong-reference-model bugs, which are the hardest to catch because the testbench looks right and runs cleanly while checking the wrong thing.

Bucket	Count	Primary cause	Prompt-fix yield
A - Prompt-Fixable	15 classes	Boilerplate forgetfulness , recurring scaffolding mistakes	High: prompt templates and a do/don't reference card close most of this
B - Architectural	6 classes	UVM phase / connection model not fully internalised	Medium: a worked-out skeleton testbench in the prompt
C - Model-Inherent	9 classes	Hallucination, comprehension limits, wrong reference models	Low: needs better models, retrieval grounding, or post-hoc validation

Table 3: Compile-failure taxonomy. The 30 observed issue classes group into three buckets with different fix-cost profiles. Bucket A dominates by frequency and is closeable with prompt engineering; bucket C is residual model-capability gap.

Functional and assertion coverage

For the coverage analysis I held the model set fixed at three: o4-mini, Gemini Pro 3.1, and Claude Opus 4.6, compared per-design against the golden original. Across all 14 designs I logged six metrics per testbench: overall coverage score, user SVA assertion pass rate, cover-property hit count, coverpoint variable closure, cross-coverage closure, and total clock cycles per assertion (a stimulus-density proxy). I then classified each design by which testbench wins on three sub-axes: best by score, best from a testing point of view, and best on assertion coverage. Claude Opus 4.6 wins the testing-point-of-view sub-axis on 11 of 14 designs by virtue of richer covergroup models, output observability, ignore_bins discipline, transition coverpoints, and user-defined cross-bins formalising protocol invariants, combined with the highest stimulus density on most designs. The original testbench wins it on the four protocol designs (APB, AHB, AXI, arbiter), where it is the only TB to use protocol-specific ignore_bins, HBURST coverage, FSM-traversal stimulus tuning, and back-to-back transition crosses. o4-mini is competitive on simple designs but fails on protocol designs (APB at 40.62%, arbiter at 21.74%), with stimulus density 10 to 100 times below Claude on every design. Gemini 3.1 produced the only complete coverage-wiring failure.

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100 % (random-only: 80.08)	~70.83 %	95.24 %	100.00 %
User SVA assertions	34 / 36 passing	34 / 36 passing	34 / 36 passing	34 / 36 passing
Cover properties	all attempted	all attempted	all attempted	all attempted
Coverpoint variables	34 / 34 = 100 %	10 / 38 (split CGs)	13 / 13 = 100 %	18 / 18 = 100 %
Cross coverage	512 / 512 = 80.08 %	0 / 0 (no cross)	10 / 13 = 76.92 %	107 / 107 = 100 %
Total clock cycles / assert	7,439	89	933	9,092

Image 4: **Adder** - Original TB vs LLM TBs Functional Coverage and Analysis

Image 4 shows the comparison of the results of the functional coverage and assertion for the adder design. Here we can see o4-mini lacked with respect to the percentage of coverpoint point variables being hit. Even though Claude shows 100% coverage, the number of coverpoint variables and cross coverage is lower than the original testbench.

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	78.91 %	57.14 %	100.00 %
User SVA assertions	7 / 9 passing	7 / 9 passing	7 / 9 passing (1 incomplete)	7 / 9 passing
Cover properties	all 6 hit	all 6 hit (low density)	all 6 hit	all 6 hit (5–80× more)
Coverpoint variables	4 / 4 = 100 %	18 / 72 = 25 %	10 / 13 = 75 %	37 / 37 = 100 %
Cross coverage	— (no cross)	— (no cross)	1 / 28 = 3.57 %	19 / 19 = 100 %
Total clock cycles / assert	390	78	696	3,384

Image 5: **USR** - Original TB vs LLM TBs Functional Coverage and Analysis

Image 5 shows a different outcome with respect to the functional coverage and assertion for the Universal Shift Register design. Here we can see o4-mini continues to lack in percentage of coverpoint point variables being hit, and there is no cross coverage defined. Claude shows 100% coverage, on a higher number of coverpoint variables and cross coverage than the original testbench.

Table 4 summarises the per-design verdict on each sub-axis. The pattern is clear: Claude dominates the testing-point-of-view sub-axis on every non-protocol design, the original dominates the protocol designs, and assertion coverage is split between Claude (on the simpler designs, by sheer stimulus

density) and the original (on designs where careful corner-case stimulus matters more than volume, sync FIFO's no-write-when-full corner, APB's ACCESS-to-IDLE FSM transition, and the arbiter).

Design	Best by score	Best from a testing PoV	Best assertion coverage
Mux	tied: orig / o4-mini / claude	Claude	Claude
Decoder	Original	Claude	Claude (close to Original)
Adder	Claude (organic)	Claude	Claude
Comparator	Original	Claude	Original (24K attempts)
ALU	Original (hollow)	Claude	Claude
Barrel shifter	Original (hollow)	Claude	Claude
Counter	tied: o4-mini / claude	Gemini 3.1 (transition bins)	Claude
USR	tied: orig / claude	Claude	Claude
FSM	Original	Claude (full-cycle seq.)	Claude
Sync FIFO	tied: orig / gem31	Claude	Original (corner stimulus)
APB	tied: orig / gem31	Claude	Original (FSM transitions)
Arbiter	Original (3 runs)	Original	Original (caught 10 fails)
AHB	Gemini 3.1	Original	Claude
AXI	Original	Original	Original

Table 4: Per-design coverage winners. For each design, the testbench that wins on each of three coverage sub-axes , raw score, depth of testing model, and assertion stimulus density and pass rate. Original = hand-written golden TB.

Future Work

First, skeleton-based prompt engineering providing a worked-out UVM testbench skeleton in the prompt with all eleven sections stubbed and TLM hookups pre-wired, can be used to close most of the prompt-fixable Category-A bucket without any change to the model, and is directly measurable against Table 2. Second, coverage-driven feedback loops which parse URG output programmatically, hand the structured summary to the LLM, accept proposed sequences only if they compile, close at least one previously-uncovered bin, and do not regress any existing bin, can be used to further automate the process of UVM testbench generation. Third, integration with LLM-generated assertions lets the LLM propose the bound SVAs as well as the testbench, a higher-stakes question that the present study deliberately controls for by using a fixed assertion set, and one that requires a separate reference-oracle infrastructure to evaluate honestly. Fourth, localised training using industry UVM testbenches, to pre-condition the LLM targets the failures that prompting alone cannot close.

Conclusion

I benchmarked three LLMs against hand-written golden testbenches across 14 RTL designs and four orthogonal evaluation axes. Three findings stand out. First, structural conformance is largely solved: Claude Opus 4.6 and o4-mini both produce eleven correctly-named UVM sections on 13 of 14 designs in a single attempt, and retry recovery closes most remaining structural failures. Second, structural correctness is necessary but not sufficient: zero of three models produced a testbench that elaborated and simulated cleanly without manual intervention, and the bulk of compile failures sit in a prompt-fixable bucket of fifteen recurring scaffolding mistakes. Third, the verification-quality gap is sharper than the structural one. Claude produces the structurally richest covergroups on most designs, but on the four bus-protocol designs, and on the only design that exposed a real bug, the hand-written original retains a meaningful lead through protocol-aware ignore_bins discipline and corner-case stimulus tuning. LLMs are credible UVM first-draft authors today; they are not yet competent verification engineers. The natural next steps are skeleton-grounded prompts to close bucket A, coverage-driven feedback loops to harden bucket B, and retrieval-grounded scoreboard reference models to chip away at bucket C.